

Exp No.: 4 Querying Ledger Data with Hyperledger Fabric

Aim

To query and retrieve ledger data from a Hyperledger Fabric blockchain network using JavaScript chaincode, and verify asset information stored in the world state database.

Software Requirements

- ☐ Kali Linux / Ubuntu
- ☐ Docker
- ☐ Docker Compose
- ☐ Node.js (Version 18 or later)
- ☐ npm
- ☐ Git
- ☐ Hyperledger Fabric Samples
- ☐ Hyperledger Fabric Binaries
- ☐ Hyperledger Fabric Docker Images
- ☐ Visual Studio Code (Optional)

Hardware Requirements

- ☐ Processor: Intel Core i3 or above
- ☐ RAM: Minimum 8 GB
- ☐ Storage: Minimum 20 GB Free Disk Space
- ☐ Internet Connection

Procedure

Step 1: Navigate to the Fabric Test Network

Open the terminal and navigate to the Hyperledger Fabric test network.

```
cd ~/fabric-dev/fabric-samples/test-network
```

Step 2: Start the Hyperledger Fabric Network

Start the Fabric network and create the application channel.

```
./network.sh up createChannel -ca
```

Expected Output

```
Creating channel 'mychannel'
```

Starting peer0.org1

Starting peer0.org2

Starting orderer.example.com

Network Started Successfully

Step 3: Deploy the Asset Transfer Chaincode

Deploy the JavaScript Asset Transfer chaincode.

```
./network.sh deployCC \  
-ccl javascript \  
-ccn basic \  
-ccp ../asset-transfer-basic/chaincode-javascript
```

Expected Output

Packaging Chaincode

Installing Chaincode

Approving Chaincode

Committing Chaincode

Chaincode committed successfully

Step 4: Initialize the Ledger

Load the ledger with sample assets.

```
peer chaincode invoke \  
-o localhost:7050 \  
--ordererTLSHostnameOverride orderer.example.com \  
--tls \  
--cafile $ORDERER_CA \  
-C mychannel \  
-n basic \  
-c '{"function":"InitLedger","Args":[]}'
```

Expected Output

Ledger Initialized Successfully

Step 5: Query All Assets

Retrieve all asset records stored in the blockchain ledger.

```
peer chaincode query \  
-C mychannel \  
-n basic \  
-c '{"Args":["GetAllAssets"]}'
```

Expected Output

```
[  
{  
  "ID":"asset1",  
  "Color":"blue",  
  "Size":5,  
  "Owner":"Tomoko",  
  "AppraisedValue":300
```

```
},  
{  
  "ID": "asset2",  
  "Color": "red",  
  
  "Size": 5,  
  "Owner": "Brad",  
  "AppraisedValue": 400  
}  
]
```

Step 6: Query a Specific Asset

Retrieve the details of asset1.

peer chaincode query \

-C mychannel \

-n basic \

-c '{"Args":["ReadAsset","asset1"]}'

Expected Output

```
{  
  "ID": "asset1",  
  "Color": "blue",  
  "Size": 5,  
  "Owner": "Tomoko",  
  "AppraisedValue": 300  
}
```

Step 7: Create a New Asset

peer chaincode invoke \

-o localhost:7050 \

--ordererTLSHostnameOverride orderer.example.com \

--tls \

--cafile \$ORDERER_CA \

-C mychannel \

-n basic \

-c '{"function":"CreateAsset","Args":["asset20","Green","8","Arun","1500"]}'

Expected Output

Asset asset20 created successfully

Step 8: Query the Newly Created Asset

peer chaincode query \

-C mychannel \

-n basic \

-c '{"Args":["ReadAsset","asset20"]}'

Expected Output

```
{
  "ID": "asset20",
  "Color": "Green",
  "Size": 8,
  "Owner": "Arun",
  "AppraisedValue": 1500
}
```

Step 9: Query All Assets Again

Retrieve all assets to verify that the new asset has been added.

```
peer chaincode query \
```

```
-C mychannel \
```

```
-n basic \
```

```
-c '{"Args":["GetAllAssets"]}'
```

Expected Output

List of all assets including asset20 displayed successfully.

Step 10: Query a Non-Existing Asset

Attempt to retrieve an asset that does not exist.

```
peer chaincode query \
```

```
-C mychannel \
```

```
-n basic \
```

```
-c '{"Args":["ReadAsset","asset50"]}'
```

Expected Output

Error: The asset asset50 does not exist.

Step 11: Stop the Fabric Network

```
./network.sh down
```

Flow of Ledger Query Process

Start Fabric Network

|

▼

Deploy Chaincode

|

▼

Initialize Ledger

|

▼

Query All Assets

|

▼

Query Specific Asset

|



Create New Asset

|



Query New Asset

|



Verify Updated Ledger

|



Stop Fabric Network

Sample Output

Fabric Network Started Successfully

Chaincode Installed Successfully

Ledger Initialized Successfully

All Assets Retrieved Successfully

Specific Asset Retrieved Successfully

New Asset Created Successfully

Ledger Updated Successfully

Query Completed Successfully

```

Session Actions Edit View Help
~n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles "$HOME/.peer/tls_rootcert_file" \
--peerAddresses localhost:9051 \
--tlsRootCertFiles "$HOME/.peer/tls_rootcert_file" \
-c '{"function":"InitLedger","Args":[]}' \
-waitForEvent
2026-08-28 18:26:18.132 15T 0001 INFO [chaincodeCmd] ClientWait → txid [7a8e8abe186fef78c7c7fb352b4210055a173b4febee37cee6558a4f5bb58ec] committed with status (VALID) at localhost:7051
2026-08-28 18:26:18.132 15T 0002 INFO [chaincodeCmd] ClientWait → txid [7a8e8abe186fef78c7c7fb352b4210055a173b4febee37cee6558a4f5bb58ec] committed with status (VALID) at localhost:9051
2026-08-28 18:26:18.132 15T 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery → Chaincode invoke successful. result: status:200

(tharun@kali): ~/fabric-dev/fabric-samples/test-network
$ peer chaincode invoke \
-o localhost:7050 \
--ordererTlsHostnamesOverride orderer.example.com \
--tls \
-cfile "$HOME/.peer/ca" \
-c mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles "$HOME/.peer/tls_rootcert_file" \
--peerAddresses localhost:9051 \
--tlsRootCertFiles "$HOME/.peer/tls_rootcert_file" \
-c '{"function":"CreateAsset","Args":["asset20","Green","8","Arun","1500"]}' \
-waitForEvent
2026-08-28 18:26:32.572 15T 0001 INFO [chaincodeCmd] ClientWait → txid [da926d1e2574b4215cd02c1bb011ac5d175180bb55f58adeadb5c3073f3da4d7] committed with status (VALID) at localhost:7051
2026-08-28 18:26:32.572 15T 0002 INFO [chaincodeCmd] ClientWait → txid [da926d1e2574b4215cd02c1bb011ac5d175180bb55f58adeadb5c3073f3da4d7] committed with status (VALID) at localhost:7051
2026-08-28 18:26:32.572 15T 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery → Chaincode invoke successful. result: status:200 payload: '{"ID":"asset20","Color":"Green","Size":8,"Owner":"Arun","AppraisedValue":1500}'

(tharun@kali): ~/fabric-dev/fabric-samples/test-network
$ peer chaincode query \
-c mychannel \
-n basic \
-c '{"function":"ReadAsset","Args":["asset20"],"Owner":"Arun","Size":8}' \
--AppraisedValue:1500,"Color":"Green","ID":"asset20","Owner":"Arun","Size":8}

(tharun@kali): ~/fabric-dev/fabric-samples/test-network
$ peer chaincode query \
-c mychannel \
-n basic \
-c '{"function":"GetAllAssets","Args":[]}' \
--AppraisedValue:200,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"},{"AppraisedValue":400,"Color":"red","ID":"asset2","Owner":"Brad","Size":5,"docType":"asset"},{"AppraisedValue":1500,"Color":"Green","ID":"asset20","Owner":"Arun","Size":8},{"AppraisedValue":500,"Color":"green","ID":"asset3","Owner":"Jin Soo","Size":10,"docType":"asset"},{"AppraisedValue":600,"Color":"yellow","ID":"asset4","Owner":"Max","Size":10,"docType":"asset"},{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"},{"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size":15,"docType":"asset"}

(tharun@kali): ~/fabric-dev/fabric-samples/test-network
$ peer chaincode query \
-c mychannel \
-n basic \
-c '{"function":"ReadAsset","Args":["asset50"]}' \
Error: endorsement failure during query. response: status:500 message:"The asset asset50 does not exist"

Session Actions Edit View Help
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vsc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org2 on channel 'mychannel'
Chaincode initialization is not required

(tharun@kali): ~/fabric-dev/fabric-samples/test-network
$ peer lifecycle chaincode querycommitted -c mychannel -n basic
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vsc, Approvals: [Org1MSP: true, Org2MSP: true]

(tharun@kali): ~/fabric-dev/fabric-samples/test-network
$ peer chaincode invoke \
-o localhost:7050 \
--ordererTlsHostnamesOverride orderer.example.com \
--tls \
-cfile "$HOME/.peer/ca" \
-c mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles "$HOME/.peer/tls_rootcert_file" \
--peerAddresses localhost:9051 \
--tlsRootCertFiles "$HOME/.peer/tls_rootcert_file" \
-c '{"function":"InitLedger","Args":[]}' \
-waitForEvent
2026-08-28 18:26:18.132 15T 0001 INFO [chaincodeCmd] ClientWait → txid [7a8e8abe186fef78c7c7fb352b4210055a173b4febee37cee6558a4f5bb58ec] committed with status (VALID) at localhost:7051
2026-08-28 18:26:18.132 15T 0002 INFO [chaincodeCmd] ClientWait → txid [7a8e8abe186fef78c7c7fb352b4210055a173b4febee37cee6558a4f5bb58ec] committed with status (VALID) at localhost:9051
2026-08-28 18:26:18.132 15T 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery → Chaincode invoke successful. result: status:200

(tharun@kali): ~/fabric-dev/fabric-samples/test-network
$ peer chaincode invoke \
-o localhost:7050 \
--ordererTlsHostnamesOverride orderer.example.com \
--tls \
-cfile "$HOME/.peer/ca" \
-c mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles "$HOME/.peer/tls_rootcert_file" \
--peerAddresses localhost:9051 \
--tlsRootCertFiles "$HOME/.peer/tls_rootcert_file" \
-c '{"function":"CreateAsset","Args":["asset20","Green","8","Arun","1500"]}' \
-waitForEvent
2026-08-28 18:26:32.572 15T 0001 INFO [chaincodeCmd] ClientWait → txid [da926d1e2574b4215cd02c1bb011ac5d175180bb55f58adeadb5c3073f3da4d7] committed with status (VALID) at localhost:7051
2026-08-28 18:26:32.572 15T 0002 INFO [chaincodeCmd] ClientWait → txid [da926d1e2574b4215cd02c1bb011ac5d175180bb55f58adeadb5c3073f3da4d7] committed with status (VALID) at localhost:7051
2026-08-28 18:26:32.572 15T 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery → Chaincode invoke successful. result: status:200 payload: '{"ID":"asset20","Color":"Green","Size":8,"Owner":"Arun","AppraisedValue":1500}'

(tharun@kali): ~/fabric-dev/fabric-samples/test-network
$ peer chaincode query \
-c mychannel \
-n basic \
-c '{"function":"ReadAsset","Args":["asset20"],"Owner":"Arun","Size":8}' \
--AppraisedValue:1500,"Color":"Green","ID":"asset20","Owner":"Arun","Size":8}

```

```

--(tharun@kali)-[~/fabric-dev/fabric-samples/test-network]
$ export PATH=$PATH:$HOME/fabric-dev/fabric-samples/bin
export FABRIC_CFG_PATH=$HOME/fabric-dev/fabric-samples/config

export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_TLS_ROOTCERT_FILE=$PWD/organizations/peerOrganizations/org1.example.com/tlsca/tlsca.org1.example.com-cert.pem
export CORE_PEER_MSPCONFIGPATH=$PWD/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051

--(tharun@kali)-[~/fabric-dev/fabric-samples/test-network]
$ export ORG2_TLS_ROOTCERT_FILE=$PWD/organizations/peerOrganizations/org2.example.com/tlsca/tlsca.org2.example.com-cert.pem

--(tharun@kali)-[~/fabric-dev/fabric-samples/test-network]
$ export ORDERER_CA=$PWD/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlsacerts/tlsca.example.com-cert.pem

--(tharun@kali)-[~/fabric-dev/fabric-samples/test-network]
$ echo $ORDERER_CA
/home/tharun/fabric-dev/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlsacerts/tlsca.example.com-cert.pem

--(tharun@kali)-[~/fabric-dev/fabric-samples/test-network]
$ ./network.sh deployCC \
  ccl javascript \
  -ccn basic \
  -csp ../asset-transfer-basic/chaincode-javascript
using docker and docker-compose
Deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: NA
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 5
- MAX_RETRY: 5
- VERBOSE: false
executing with the following
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- ['false' = true '']
peer lifecycle chaincode package basic.tar.gz --path ../asset-transfer-basic/chaincode-javascript --lang node --label basic_1.0
peer lifecycle chaincode install basic
chaincode is packaged
Installing chaincode on peer0.org1...
Joining organization 1
peer lifecycle chaincode queryinstalled --output json
jq -r '.try (.installed_chaincodes[]|.package_id)'
grep "basic_1.0:1d4d445573112813889f87c307bc2b5f8e664c07b24c035bee15cbcc7f59b6295"

```

Result

The ledger data stored in the Hyperledger Fabric blockchain network was successfully queried using JavaScript chaincode. Asset records were retrieved from the world state database, specific asset details were verified, and newly created assets were successfully queried, demonstrating efficient data retrieval and verification in a permissioned blockchain environment.